МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена трудового Красного Знамени федеральное государственное бюджетное
образовательное учреждение высшего образования
«Московский технический университет связи и информатики»

Кафедра Математическая кибернетика и информационные технологии

Отчет по лабораторной работе №9

Выполнил студент группы:

БПИ2401

Мещеряков Кирилл Владимирович

Руководитель:

Харрасов Камиль Раисович

Москва, 2026

Цель работы

Изучить инструмент автоматизации сборки Apache Maven: освоить создание Maven-проекта, настройку файла `pom.xml`, управление зависимостями и запуск жизненного цикла сборки.

Задание

Создать Maven-проект на языке Java, реализующий иерархию классов **«Животные»**. Проект должен содержать:

- абстрактный базовый класс с общими полями и методами;
- классы-наследники с переопределёнными методами;
- демонстрационный `main`-метод;
- зависимости и плагины, описанные в `pom.xml`.

Описание проекта

Структура проекта

```
├── pom.xml
└── src/
    └── main/
        └── java/
            └── org/example/
                ├── Animal.java
                ├── Cat.java
                ├── Parrot.java
                ├── Fish.java
                └── Main.java
```

Файл `pom.xml`

Файл описывает координаты проекта, зависимости и плагины сборки.

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
                            http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>org.example</groupId>
    <artifactId>lab9</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <maven.compiler.source>17</maven.compiler.source>
        <maven.compiler.target>17</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

    <dependencies>
        <dependency>
            <groupId>junit</groupId>
            <artifactId>junit</artifactId>
            <version>4.13.2</version>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
```

```
            <artifactId>maven-compiler-plugin</artifactId>
            <version>3.8.1</version>
        </plugin>
        <plugin>
            <groupId>org.codehaus.mojo</groupId>
            <artifactId>exec-maven-plugin</artifactId>
            <version>3.1.0</version>
            <configuration>
                <mainClass>org.example.Main</mainClass>
            </configuration>
        </plugin>
    </plugins>
  </build>
</project>
```

Ключевые элементы:

- `groupId / artifactId / version` — координаты артефакта (GAV).
- `dependencies` — зависимость JUnit 4 с областью `test`.
- `maven-compiler-plugin` — компиляция исходников под Java 17.
- `exec-maven-plugin` — запуск класса `Main` командой `exec:java`.

Исходный код

Абстрактный класс `Animal`

```
package org.example;

public abstract class Animal {
  private String name;
```

```
    private int age;

    public Animal(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public String getName() { return name; }
    public int getAge()     { return age; }

    public abstract void speak();

    public void move() {
        System.out.println(name + " двигается.");
    }
}
```

Класс `Cat`

```
package org.example;

public class Cat extends Animal {
    public Cat(String name, int age) {
        super(name, age);
    }

    @Override
    public void speak() {
        System.out.println(getName() + " говорит: Мяу!");
    }
}
```

Класс `Parrot`

```
package org.example;

public class Parrot extends Animal {
    public Parrot(String name, int age) {
        super(name, age);
    }

    @Override
    public void speak() {
        System.out.println(getName() + " говорит: Привет! Карр!");
    }

    @Override
    public void move() {
        System.out.println(getName() + " летает.");
    }
}
```

Класс `Fish`

```
package org.example;

public class Fish extends Animal {
    public Fish(String name, int age) {
        super(name, age);
    }

    @Override
    public void speak() {
```

```
        System.out.println(getName() + " молчит...");
    }

    @Override
    public void move() {
        System.out.println(getName() + " плавает.");
    }
}
```

Класс `Main`

```
package org.example;

public class Main {
    public static void main(String[] args) {
        Animal cat    = new Cat("Кот 1", 3);
        Animal parrot = new Parrot("Попугай 1", 2);
        Animal fish   = new Fish("Рыба 1", 1);

        System.out.println("Кот: " + cat.getName() + ", возраст: " + cat.getAge());
        cat.speak();
        cat.move();

        System.out.println("Попугай: " + parrot.getName() + ", возраст: " +
parrot.getAge());
        parrot.speak();
        parrot.move();

        System.out.println("Рыбка: " + fish.getName() + ", возраст: " +
fish.getAge());
        fish.speak();
```

```
        fish.move();
    }
}
```

Запуск проекта

Для сборки и запуска выполняется команда:

mvn clean install exec:java

Фаза / Цель	Действие
`clean`	Удаление директории `target`
`install`	Компиляция, тест, упаковка, установка в `.m2`
`exec:java`	Запуск класса `org.example.Main`

Результат выполнения программы

Кот: Кот 1, возраст: 3

Кот 1 говорит: Мяу!

Кот 1 двигается.

Попугай: Попугай 1, возраст: 2

Попугай 1 говорит: Привет! Карр!

Попугай 1 летает.

Рыбка: Рыба 1, возраст: 1

Рыба 1 молчит...

Рыба 1 плавает.

Ответы на контрольные вопросы

1. **Что такое Apache Maven и для чего он используется?**

Apache Maven — инструмент автоматизации сборки Java-проектов. Он стандартизирует процесс разработки: управляет зависимостями, выполняет компиляцию, тестирование, упаковку и публикацию артефактов.

2. **Как установить Maven на различные ОС?**

- *Windows*: скачать архив с сайта maven.apache.org, распаковать, добавить путь к `bin/` в переменную `PATH`.

-*Linu*: `sudo apt install maven`.

- *macOS*: `brew install maven`.

3. **Какова структура проекта Maven?**

`pom.xml` в корне; `src/main/java` — исходники; `src/main/resources` — ресурсы; `src/test/java` — тесты; `target/` — результаты сборки.

4. **Что такое POM-файл и какова его роль?**

POM (Project Object Model) — XML-файл `pom.xml`, содержащий описание проекта: координаты (GAV), зависимости, плагины, настройки компилятора. Это главный управляющий файл проекта.

5. **Какова структура файла POM?**

Корневой элемент `<project>`, обязательные `<modelVersion>`, `<groupId>`, `<artifactId>`, `<version>`, необязательные `<dependencies>`, `<build>`, `<properties>`, `<profiles>`.

6. **Что такое зависимости в Maven?**

Внешние библиотеки (jar-файлы), добавляемые в секции `<dependencies>` с указанием GAV-координат. Maven скачивает их из репозиториев автоматически.

7. **Виды репозиториев Maven?**

Локальный (`~/.m2/repository`), *центральный* (Maven Central), *удалённый* (корпоративные Nexus/Artifactory или сторонние).

8. **Как добавить зависимость?**

Добавить блок `<dependency>` с `<groupId>`, `<artifactId>`, `<version>` (и при необходимости `<scope>`) внутрь `<dependencies>` в `pom.xml`.

9. **Что такое плагины Maven?**

Модули, расширяющие возможности Maven (компиляция, запуск тестов, упаковка). Объявляются в `<build><plugins>`. Примеры: `maven-compiler-plugin`, `exec-maven-plugin`.

10. **Как создать новый проект Maven?**

```
mvn archetype:generate -DgroupId=com.example -DartifactId=my-app \
    -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
```

11. **Фазы и цели Maven — в чём отличие?**

Фаза — этап жизненного цикла (`compile`, `test`, `package`). *Цель* — конкретная задача плагина (`compiler:compile`). Фаза запускает привязанные к ней цели.

12. **Как выполнить сборку проекта?**

`mvn clean install` — очищает `target/`, компилирует, запускает тесты, упаковывает и устанавливает артефакт в локальный репозиторий.

13. **Жизненный цикл сборки Maven:**

Три цикла: `default` (сборка), `clean` (очистка), `site` (документация). Цикл `default` включает фазы: `validate → compile → test → package → verify → install → deploy`.

14. **Профили в Maven**

Объявляются в `<profiles>` в `pom.xml`. Активируются командой `mvn package -P <profileId>`. Позволяют менять конфигурацию под разные среды (dev, prod).

15. **Управление версиями зависимостей:**

Через тег `<version>` в каждой зависимости. В многомодульных проектах — через `<dependencyManagement>` в родительском POM для централизованного управления.

16. **Что такое SNAPSHOT-версии?**

SNAPSHOT означает нестабильную версию в разработке (`1.0-SNAPSHOT`). Maven регулярно проверяет обновления таких версий, в отличие от релизных.

17. **Maven и отчёты о качестве кода:**

Подключаются плагины `maven-checkstyle-plugin`, `jacoco-maven-plugin`, `spotbugs-maven-plugin` в секцию `<reporting>` и запускаются фазой `mvn site`.

18. **Основные команды Maven:**

Команда	Действие
`mvn clean`	Удалить `target/`
`mvn compile`	Скомпилировать исходники
`mvn test`	Запустить тесты
`mvn package`	Упаковать в JAR/WAR
`mvn install`	Установить в локальный репозиторий

19. **Интеграция Maven с Git:**

В `.gitignore` исключается директория `target/`. Для автоматизации релизов используется `maven-release-plugin`.

20. **Сторонний репозиторий в pom.xml:**

```xml
<repositories>
    <repository>
        <id>my-repo</id>
        <url>https://repo.example.com/maven2</url>
    </repository>
</repositories>
```

Выводы

В ходе лабораторной работы был изучен инструмент автоматизации сборки Apache Maven. Создан Maven-проект с иерархией классов «Животные», настроен файл `pom.xml` с зависимостями и плагинами. Освоены основные команды Maven, изучены понятия POM, репозитория, зависимостей, плагинов, фаз и целей жизненного цикла сборки.